MATHEMATICS FOR MACHINE LEARNING

CampusX · Data Science Mentorship Program 2023 · Video 21 of 23

# SESSION 21
# Singular Value Decomposition (SVD)

Full & Truncated SVD · Geometric Transformations · Matrix Approximation · Applications

- SVD Definition & Formula
- Left & Right Singular Vectors
- Singular Values & Diagonal Matrix
- Geometric Intuition of SVD
- SVD vs Eigendecomposition
- Low-Rank Matrix Approximation
- Image Compression with SVD
- Python SVD Implementation

**Session Notes — Complete Reference**

Playlist: Maths for Machine Learning · Instructor: Nitish Singh (CampusX)

# 📑 Table of Contents

# 01 · SVD Definition & Formula

**Singular Value Decomposition (SVD)** generalises eigendecomposition to any $m imes n$ matrix:

**SVD MATRIX FACTORIZATION**

$A = U \cdot \Sigma \cdot V^T$ where: $U$ is $m \times m$ orthogonal matrix (left singular vectors) $\Sigma$ is $m \times n$ diagonal matrix containing singular values $V$ is $n \times n$ orthogonal matrix (right singular vectors)

# 02 · Left & Right Singular Vectors

- **Right Singular Vectors ($V$):** Eigenvectors of the symmetric matrix $A^TA$.
- **Left Singular Vectors ($U$):** Eigenvectors of the symmetric matrix $AA^T$.

# 03 · Singular Values & Diagonal Matrix

The diagonal entries of $\Sigma$ are the **singular values** $\sigma_i$, which are the square roots of the eigenvalues of $A^TA$ (or $AA^T$). They are non-negative and sorted in descending order.

# 04 · Geometric Intuition of SVD

SVD states that any linear transformation can be broken down into three sequential steps: a rotation ($V^T$), a scaling along the coordinate axes ($\Sigma$), and a second rotation ($U$).

# 05 · SVD vs Eigendecomposition

| FEATURE | EIGENDECOMPOSITION | SVD |
|---|---|---|
| **Applicability** | Square matrices only ($n imes n$) | Any matrix ($m imes n$) |
| **Matrix properties** | Not always diagonalizable | Always exists for any real matrix |
| **Orthogonality** | Eigenvectors not always orthogonal | Singular vectors are always orthogonal |

# 06 · Low-Rank Matrix Approximation

The **Eckart-Young-Mirsky Theorem** states that we can form the optimal rank-$k$ approximation of a matrix $A$ by keeping only the top $k$ singular values and corresponding vectors: $A_k = \sum_{i=1}^k \sigma_i \mathbf{u_i} \mathbf{v_i}^T$. This is the basis of data compression.

# 07 · Image Compression with SVD

An image is represented as a matrix. Reconstructing the image using only the top $k$ singular values ($k \ll \min(m,n)$) compresses the image size while retaining most visual details.

# 08 · Python SVD Implementation

```
import numpy as np

A = np.array([[1, 2], [3, 4], [5, 6]])

U, s, Vt = np.linalg.svd(A)
print("U matrix:
", U)
print("Singular values:", s)
print("V^T matrix:
", Vt)
```

# 09 · Quick Reference Card

### 🚀 SVD Cheat Sheet

- **Formula:** $A = U\Sigma V^T$.
- **Relationship:** Singular values = $\sqrt{\text{Eigenvalues of } A^TA}$.
- **Low-rank representation:** Drop smaller singular values to filter out noise.